

16-831: Introduction to Robot Learning, Fall 2024

Q Learning and Variants



Generated by GPT-4 with DALLÉ

Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Late Policy

- ☐ Reminder: Everyone has five late days
- ☐ Only for homework assignments

Late Policy

For the homework assignments only, students will be allowed a total of five late days per semester. Any work submitted late after the five late days have been used will be given an automatic zero on the assignment. Make sure to start early and complete your assignments on time! Please note that the late days do not apply to any part of the final project. This policy will be enforced strictly.

Course Structure

Latest schedule: <https://16-831.github.io/fall24/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☐ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☒ Topic group 4: Model-free reinforcement learning
- ☐ Topic group 5: Model-based reinforcement learning
- ☐ Topic group 6: Bandit, preference-based learning, exploration
- ☐ Topic group 7: Reinforcement learning from offline data (one guest lecture)
- ☐ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation

Outline

- ❑ Recap: Policy/Value Iterations
 - Q-Value Iteration
- ❑ Fitted Q-Iteration (FQI)
- ❑ Online Q-Learning
- ❑ Deep Q-Network (DQN)
- ❑ Improvements and extensions
 - DDQN
 - Prioritized experience replay
 - Dueling architecture
 - Q-learning with continuous actions

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Outline

- ❑ Recap: Policy/Value Iterations
 - Q-Value Iteration
- ❑ Fitted Q-Iteration (FQI)
- ❑ Online Q-Learning
- ❑ Deep Q-Network (DQN)
- ❑ Improvements and extensions
 - DDQN
 - Prioritized experience replay
 - Dueling architecture
 - Q-learning with continuous actions

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Recap: Policy Iteration

- Can also do Q value function (policy evaluation is more complex but improvement is easier)

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \operatorname{argmax}_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ false

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Recap: Value Iteration

- ❑ Policy iteration involves an iterative policy evaluation step
- ❑ Can we remove it?
- ❑ Yes! Think about Bellman optimality equation:

$$V^*(s) = \max_a Q^*(s, a) = \max_a \sum_{s'} p(s', r|s, a)[r + \gamma \cdot V^*(s')]$$

- ❑ Idea: directly update the value function!

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \arg \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma V(s')]$$

Summary: Policy/Value Iterations

- ❑ Bellman optimality equations are **nonlinear**
- ❑ In **tabular case with known dynamics**, we can do DP-based policy improvement
- ❑ In general RL settings, **approximately** solve the Bellman optimality equations!
- ❑ Today: $p(s'|s, a)$ is unknown!

$$V^*(s) = \max_a \sum_{s', r} p(s', r|s, a) [r + \gamma \cdot V^*(s')]$$

$$Q^*(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma \cdot \max_{a'} Q^*(s', a')]$$

continuous state/action space (need
function approximation for value
functions)

Unknown dynamics
(must be approximated,
usually using sampling)

max over continuous
action space

Q-Value Iteration

- ❑ Will converge to Q^* because of the **contraction** (although this equation is **nonlinear**)

$$Q_{k+1}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot \max_{a'} Q_k(s', a')] , \forall s, a$$

- ❑ Need to know $p(s', r | s, a)$ for dynamic programming

- ❑ Algorithm:

- Initialize Q_0
- $Q_{k+1} = \mathfrak{B}Q_k$ ($\mathfrak{B}$ is called the Bellman operator)
- Once converged, $\pi(s) = \underset{a}{\operatorname{argmax}} Q_K(s, a)$

- ❑ What if we don't know $p(s', r | s, a)$?

- Sampling from interacting with the environment!

- ❑ What if $|S|$ and $|A|$ are large, or (s, a) is in continuous space?

- Function approximation!

Q-Value Iteration

❑ Will converge to Q^* because of the **contraction**

$$Q_{k+1}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot \max_{a'} Q_k(s', a')] , \forall s, a$$

❑ The size of $Q(s, a)$: $|S| \times |A|$!

- How many states in an Atari game?
- It is called “**curse of dimensionality**”! (from Richard Bellman)



$$|\mathcal{S}| = (255^3)^{200 \times 200}$$

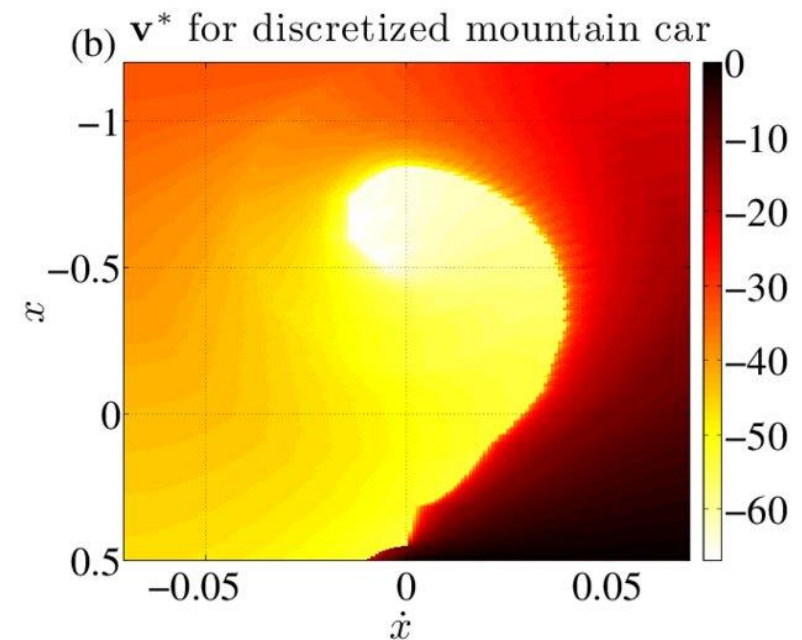
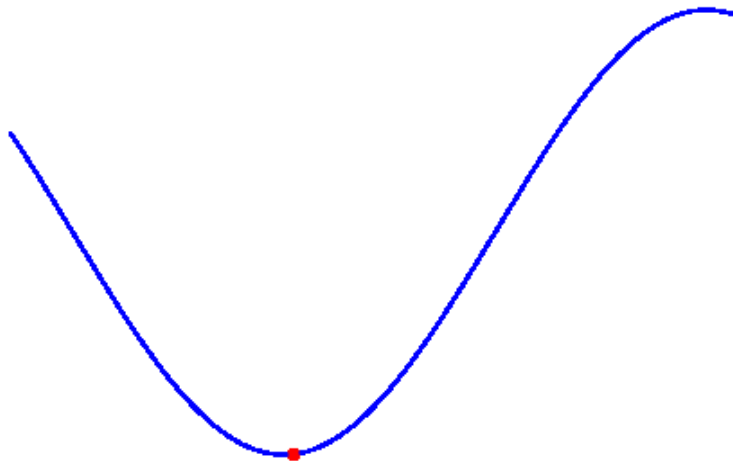


Q-Value Iteration

□ Will converge to Q^* because of the **contraction**

$$Q_{k+1}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot \max_{a'} Q_k(s', a')] , \forall s, a$$

- Good news: $Q(s, a)$ has some “patterns”!
- That is exactly what **deep learning** is good at!



Outline

- ❑ Recap: Policy/Value Iterations
 - Q-Value Iteration
- ❑ Fitted Q-Iteration (FQI)
- ❑ Online Q-Learning
- ❑ Deep Q-Network (DQN)
- ❑ Improvements and extensions
 - DDQN
 - Prioritized experience replay
 - Dueling architecture
 - Q-learning with continuous actions

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Fitted Q Iteration (FQI)

□ Standard Q-value iteration:

$$Q_{k+1}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot \max_{a'} Q_k(s', a')] , \forall s, a$$

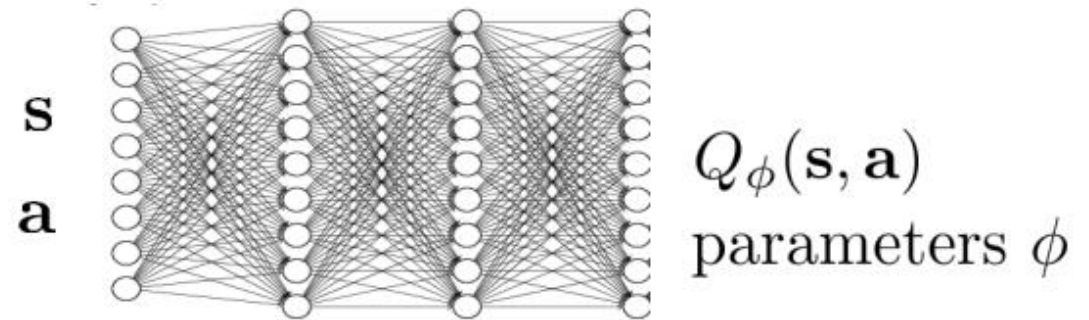
□ Fitted Q Iteration:

- Collect dataset $\{(s_i, a_i, s'_i, r_i)\}$ using “some policy” (not necessarily optimal!)

- Repeat:

- $y_i \leftarrow r(s_i, a_i) + \gamma \max_{a'_i} Q_\phi(s'_i, a'_i)$

- $\phi \leftarrow \operatorname{argmin}_{\phi} \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2$



□ Works for **off-policy** samples!

□ Convergence guarantee under some assumptions

- “Some policy” has good “coverage”
- Realizability: Q_ϕ can represent Q^*
- ϕ has some good property

Fitted Q Iteration (FQI)

❑ Standard Q-value iteration:

$$Q_{k+1}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot \max_{a'} Q_k(s', a')] , \forall s, a$$

❑ Fitted Q Iteration:

- Collect dataset $\{(s_i, a_i, s'_i, r_i)\}$ using “some policy”
- Repeat:
 - $y_i \leftarrow r(s_i, a_i) + \gamma \max_{a'_i} Q_\phi(s'_i, a'_i)$
 - $\phi \leftarrow \operatorname{argmin}_{\phi} \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2$

❑ Main idea 1: using samples $\{(s_i, a_i, s'_i, r_i)\}$ to approximate the dynamic programming step

❑ Main idea 2: using function approximation Q_ϕ

Fitted Q Iteration (FQI)

□ Fitted Q Iteration:

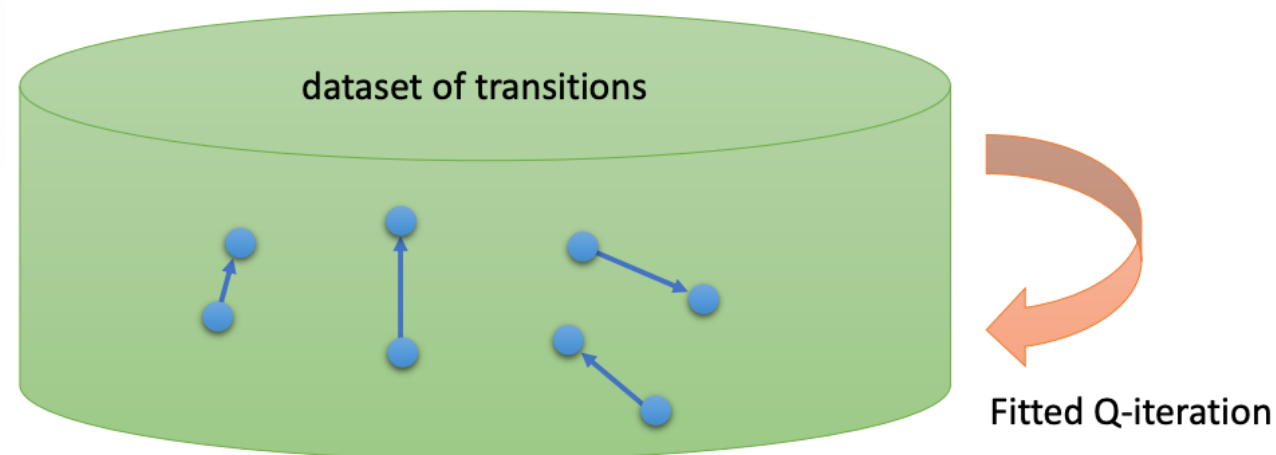
- Collect dataset $\{(s_i, a_i, s'_i, r_i)\}$ using “some policy” π
- Repeat:
 - $y_i \leftarrow r(s_i, a_i) + \gamma \max_{a'_i} Q_\phi(s'_i, a'_i)$
 - $\phi \leftarrow \operatorname{argmin}_{\phi} \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2$

□ Why is FQI **off-policy**?

- Given s_i, a_i , this update is independent of π !

$$y_i \leftarrow r(s_i, a_i) + \gamma \max_{a'_i} Q_\phi(s'_i, a'_i)$$

- π only influences data distribution!



Fitted Q Iteration (FQI)

□ Fitted Q Iteration:

- Collect dataset $\{(s_i, a_i, s'_i, r_i)\}$ using “some policy” π
- Repeat:
 - $y_i \leftarrow r(s_i, a_i) + \gamma \max_{a'_i} Q_\phi(s'_i, a'_i)$
 - $\phi \leftarrow \operatorname{argmin}_{\phi} \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2$

□ Why FQI makes sense? It minimizes the following error:

$$\mathbb{E}_{(s,a,s',r) \sim \pi,p} \left[\left(Q_\phi(s, a) - r(s, a) - \gamma \max_{a'} Q_\phi(s', a') \right)^2 \right]$$

□ Recall Bellman optimality equation:

$$Q^*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot \max_{a'} Q^*(s', a')]$$

Fitted Q Iteration (FQI)

□ Fitted Q Iteration:

- Collect dataset $\{(s_i, a_i, s'_i, r_i)\}$ using “some policy” π
- Repeat:
 - $y_i \leftarrow r(s_i, a_i) + \gamma \max_{a'_i} Q_\phi(s'_i, a'_i)$
 - $\phi \leftarrow \operatorname{argmin}_\phi \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2$

□ Problem of FQI:

- It is an offline algorithm ($\{(s_i, a_i, s'_i, r_i)\}$ is pre-collected and fixed)
 - If the data collection policy is too random: inefficient learning
 - If the data collection policy is too deterministic: bad coverage
- The optimization $\operatorname{argmin}_\phi \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2$ is hard!

From FQI to Online Q-Learning

❑ Fitted Q Iteration:

- Collect dataset $\{(s_i, a_i, s'_i, r_i)\}$ using “some policy” π
- Repeat:
 - $y_i \leftarrow r(s_i, a_i) + \gamma \max_{a'_i} Q_\phi(s'_i, a'_i)$
 - $\phi \leftarrow \operatorname{argmin}_\phi \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2$

❑ Online Q-Learning. Repeat (online):

- Take “some” action a and observe (s, a, s', r)
- $y \leftarrow r + \gamma \max_{a'} Q_\phi(s', a')$
- Update: $\phi \leftarrow \phi - \eta (Q_\phi(s, a) - y) \nabla_\phi Q_\phi(s, a)$

❑ Questions:

- How to design the data collection policy? How to explore?
- Can we do a batch update rather than single data point?
- Does it converge/work?

Online Q-Learning

□ **Online Q-Learning.** Repeat (online):

- Take “some” action a and observe (s, a, s', r)
- Update: $\phi \leftarrow \phi - \eta \left(Q_\phi(s, a) - r - \gamma \max_{a'} Q_\phi(s', a') \right) \nabla_\phi Q_\phi(s, a)$

□ Q learning is **NOT** gradient descent (GD)!

- $y = r(s, a) + \gamma \max_{a'} Q_\phi(s', a')$ is fixed
- Full GD will consider $\nabla_\phi y$

□ It is a **semi-gradient** method

- Theoretical analysis is hard ...

□ But we can make it work well in practice!

Outline

- ❑ Recap: Policy/Value Iterations
 - Q-Value Iteration
- ❑ Fitted Q-Iteration (FQI)
- ❑ Online Q-Learning
- ❑ Deep Q-Network (DQN)
- ❑ Improvements and extensions
 - DDQN
 - Prioritized experience replay
 - Dueling architecture
 - Q-learning with continuous actions

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Deep Q-Learning

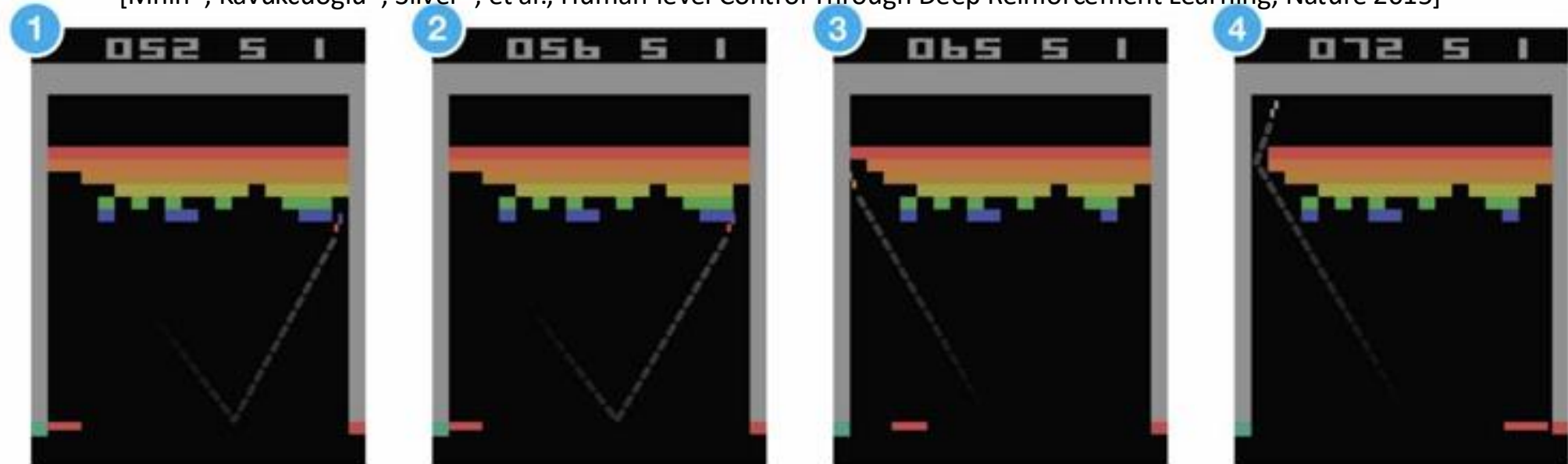
❑ DQN: Deep Q-Networks

- DeepMind showed in a 2013 workshop paper (later a Nature 2015 paper) how to train an agent to play Atari from *raw pixels* and *environment reward*.

❑ Improvements and extensions: Double, Prioritized, Dueling, DDPG, ...

[Mnih et al., Playing Atari with Deep Reinforcement Learning, NeurIPS workshop 2013]

[Mnih*, Kavukcuoglu*, Silver*, et al., Human-level Control Through Deep Reinforcement Learning, Nature 2015]



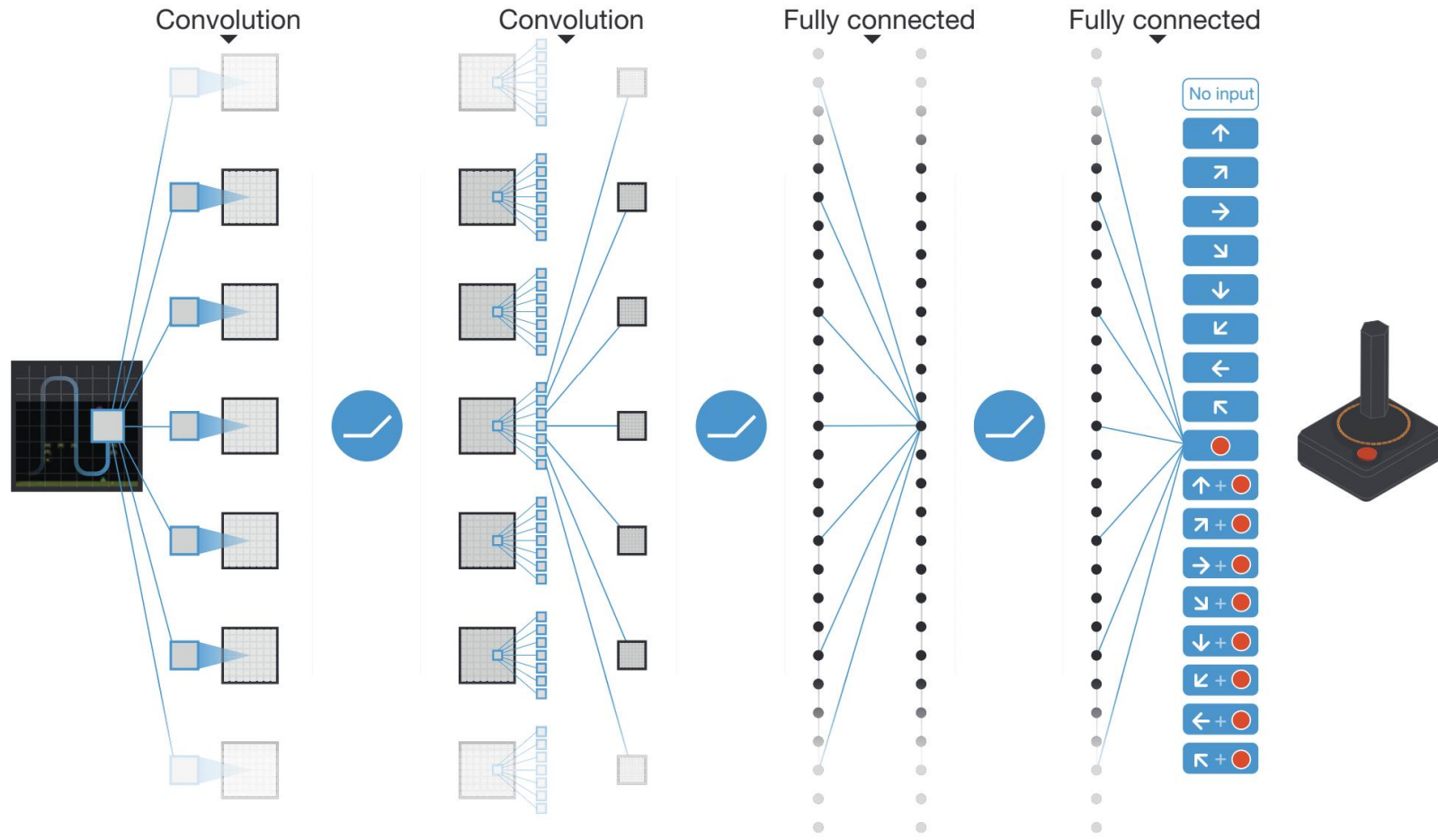
DQN: Deep Q-Networks



Deep Q-Network

□ Q network:

- Input: consecutive game frames (4 for Atari)
- Output: Q values for all actions (discrete)



$$\begin{bmatrix} Q_{\theta}(\mathbf{s}, \mathbf{a}^{(1)}) \\ Q_{\theta}(\mathbf{s}, \mathbf{a}^{(2)}) \\ \vdots \\ Q_{\theta}(\mathbf{s}, \mathbf{a}^{(k)}) \end{bmatrix}$$

DQN: Epsilon Greedy

□ *Epsilon-greedy* policy with a decaying epsilon for exploration:

$$\pi_{\theta}(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 - \epsilon & \text{if } \mathbf{a} = \operatorname{argmax}_{\mathbf{a} \in \mathcal{A}} Q_{\theta}(\mathbf{s}, \mathbf{a}) \\ \frac{\epsilon}{|\mathcal{A}| - 1} & \text{otherwise} \end{cases}$$

□ Intuition of epsilon-greedy:

- Early in training: Q-values largely a guess, explore to see rewards
- Later in training: Q-values mostly stable, might be better to exploit

□ Vanilla algorithm:

- Epsilon greedy policy gives a and observe (s, a, s', r)
- Update: $\phi \leftarrow \phi - \eta \left(Q_{\phi}(s, a) - r - \gamma \max_{a'} Q_{\phi}(s', a') \right) \nabla_{\phi} Q_{\phi}(s, a)$

□ BUT...

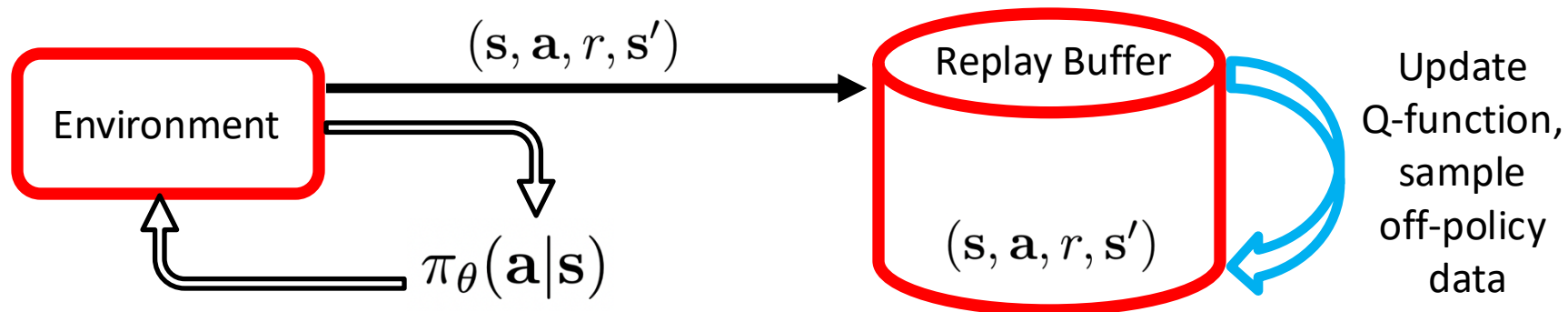
- Correlated samples
- It is NOT GD
- Target value $r + \gamma \max_{a'} Q_{\phi}(s', a')$ always changes!

DQN: Replay Buffer

All algorithmic ideas in DQN (and variants) aim to make Q learning “closer” to standard supervised learning with SGD update

❑ Idea 1: **experience replay**

- Fundamental reason: Q-learning is **off-policy**!

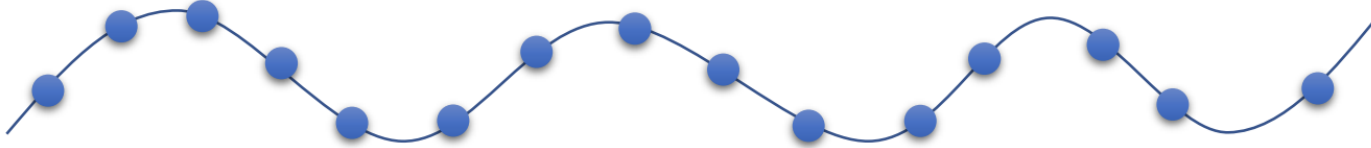


❑ Q learning with replay buffer algorithm:

- Epsilon greedy policy gives a and observe (s, a, s', r)
- Update the replay buffer
- Sample a batch from the buffer and compute the target y_i
- Update: $\phi \leftarrow \phi - \eta \sum_i (Q_{\phi}(s_i, a_i) - y_i) \nabla_{\phi} Q_{\phi}(s_i, a_i)$

DQN: Target Networks for Stable Regression

❑ However, the target is still “moving”



❑ Compare offline FQI and online Q learning:

- Fitted Q iteration (stable regression target in each epoch!):

- $y_i \leftarrow r(s_i, a_i) + \gamma \max_{a'_i} Q_\phi(s'_i, a'_i)$

- $\phi \leftarrow \operatorname{argmin}_{\phi} \sum_i \|Q_\phi(s_i, a_i) - y_i\|^2$

- Q learning (one semi-gradient step, moving target!):

- $\phi \leftarrow \phi - \eta \left(Q_\phi(s, a) - r - \gamma \max_{a'} Q_\phi(s', a') \right) \nabla_\phi Q_\phi(s, a)$

❑ Idea 2: **target network**

DQN: Target Networks for Stable Regression

□ Idea 2: **target network**

- Two Q-networks, with the same architecture but a different set of parameters.
- Periodically update the “lagging/target” parameters: $\phi^- \leftarrow \phi$

$$\mathbb{E}_{(s,a,s',r)} \left[\left(Q_{\phi}(s,a) - r(s,a) - \gamma \max_{a'} Q_{\phi}(s',a') \right)^2 \right]$$
$$\mathbb{E}_{(s,a,s',r)} \left[\left(Q_{\phi}(s,a) - r(s,a) - \gamma \max_{a'} Q_{\phi^-}(s',a') \right)^2 \right]$$

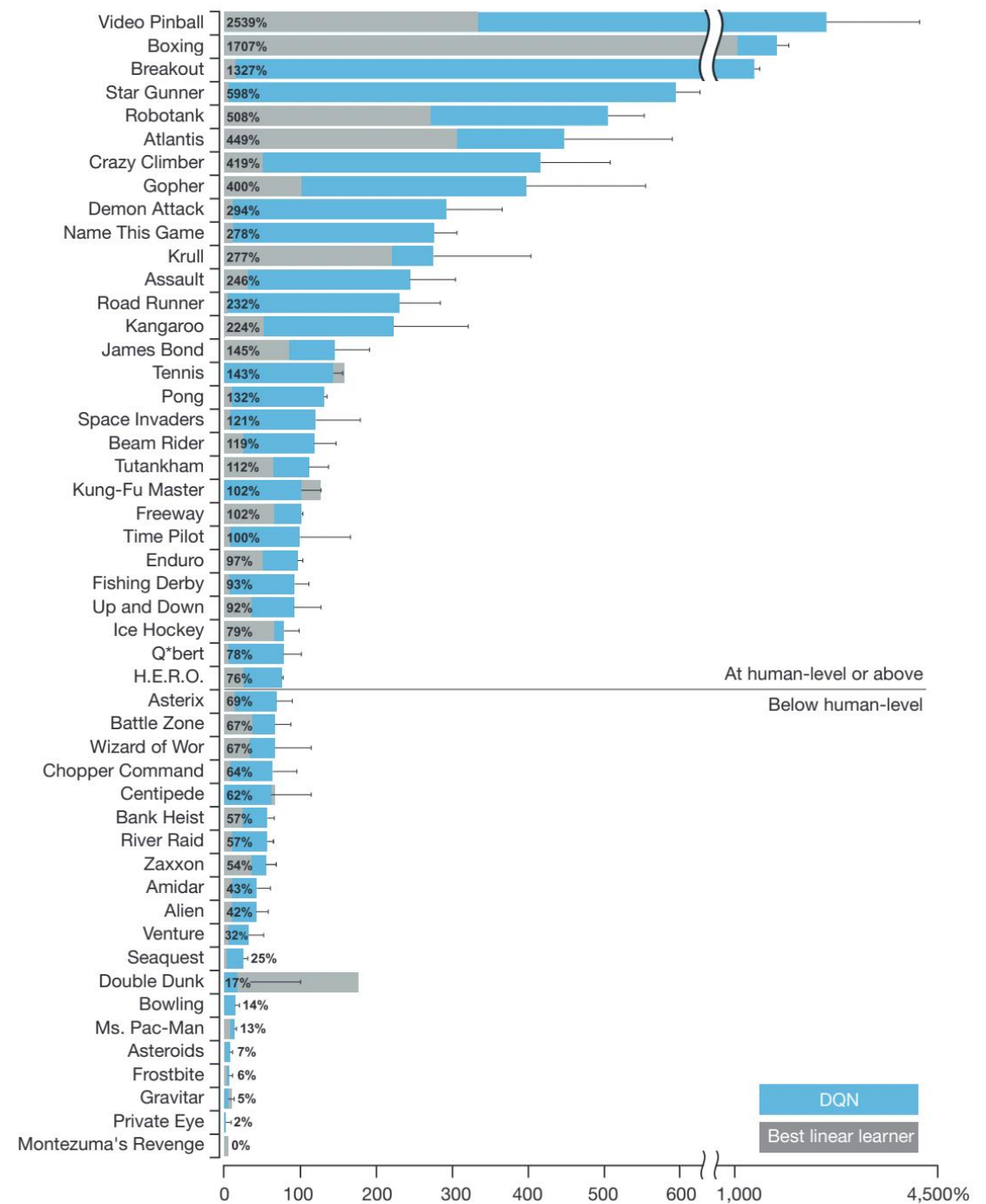
DQN: Idea 1 + Idea 2

Repeat:

- Epsilon greedy policy gives a and observe (s, a, s', r)
- Update the replay buffer
- Sample a batch from the buffer and compute the target $y_i = r(s_i, a_i) + \gamma \max_{a'_i} Q_{\phi^-}(s'_i, a'_i)$
- Update: $\phi \leftarrow \phi - \eta \sum_i (Q_{\phi}(s_i, a_i) - y_i) \nabla_{\phi} Q_{\phi}(s_i, a_i)$
- Periodically update ϕ^- : $\phi^- \leftarrow \phi$

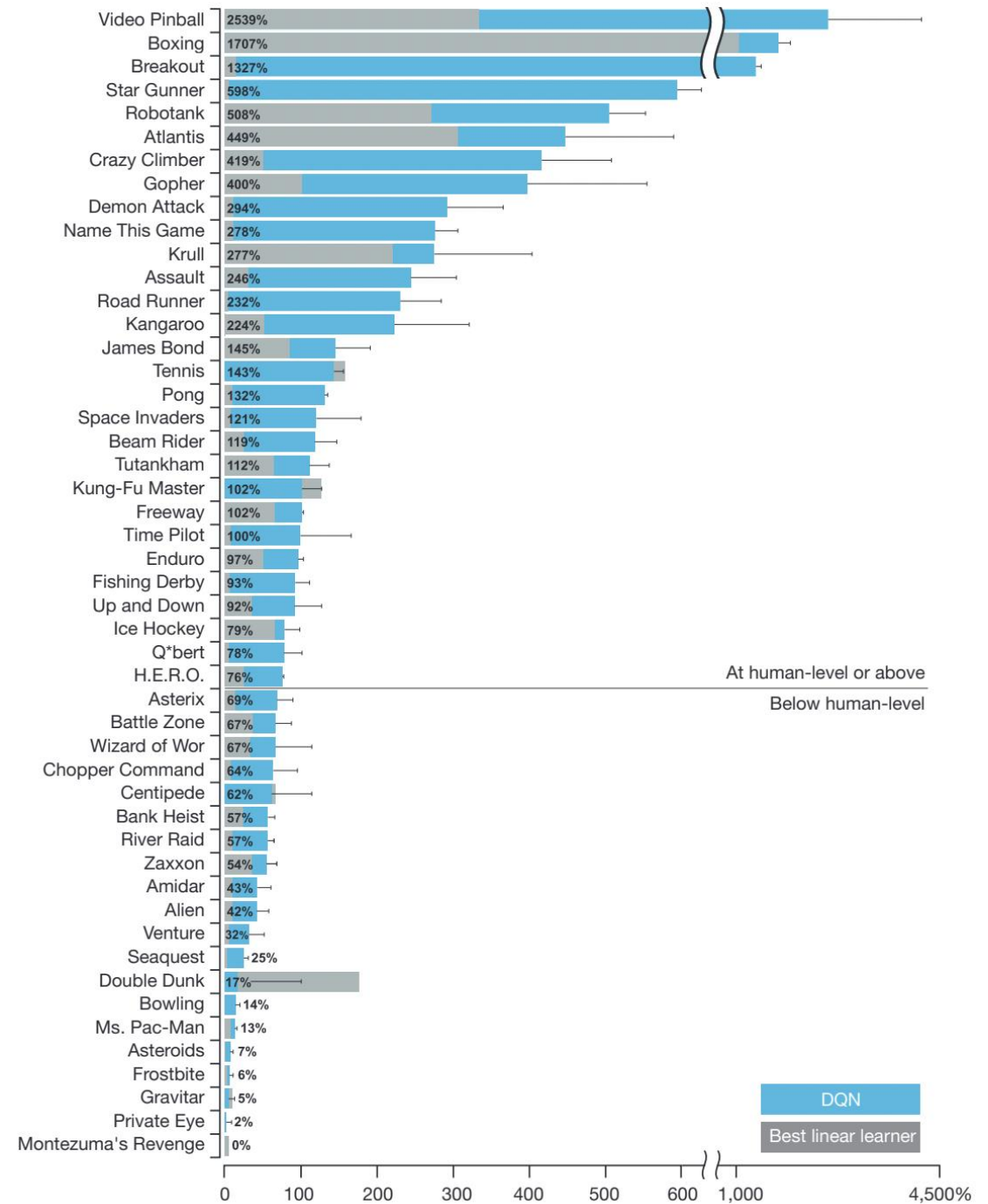
□ Alternative target network:

- $\phi^- \leftarrow (1 - \lambda)\phi + \lambda\phi^-$
- E.g., $\lambda = 0.999$



DQN: Practical Tips

- ❑ It can take a while to converge (be patient).
- ❑ High exploration initially, then reduce.
- ❑ Test on known / easy tasks first.
 - Pong and Breakout are good options.
 - Avoid sparse-reward tasks such as Montezuma's Revenge.
- ❑ Use / add more advanced features, as we'll soon cover.
- ❑ Use a well tested library (e.g., stable-baselines3).
- ❑ The best way to *learn* about algorithms is to implement them!



Outline

- ❑ Recap: Policy/Value Iterations
 - Q-Value Iteration
- ❑ Fitted Q-Iteration (FQI)
- ❑ Online Q-Learning
- ❑ Deep Q-Network (DQN)
- ❑ Improvements and extensions
 - DDQN
 - Prioritized experience replay
 - Dueling architecture
 - Q-learning with continuous actions

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

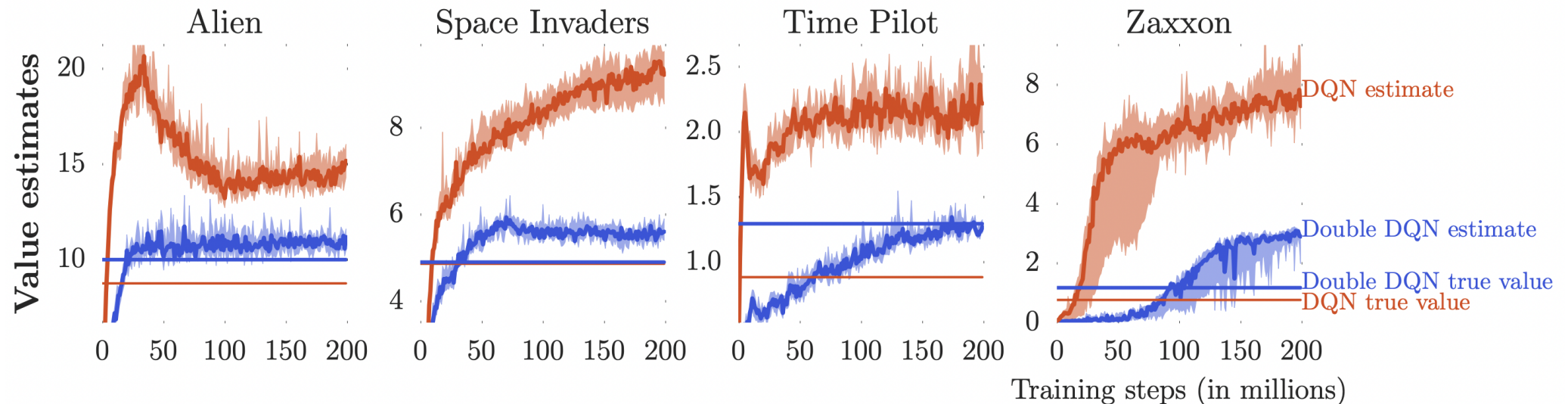
Overestimation in Q-Learning

□ The red term is a “guess of guess”

$$\mathbb{E}_{(s,a,s',r)} \left[\left(Q_{\phi}(s,a) - \underset{a'}{r(s,a)} - \gamma \underset{a'}{\max} Q_{\phi-}(s',a') \right)^2 \right]$$

□ $r(s,a) + \gamma \underset{a'}{\max} Q_{\phi-}(s',a') = r + \gamma \underset{a'}{Q_{\phi-}}(s', \underset{a'}{\operatorname{argmax}} Q_{\phi-}(s',a'))$

- Argmax amplifies the noise in $Q_{\phi-}$, which is imperfect
- Then we evaluate it using the same Q function!



Double Q-Learning

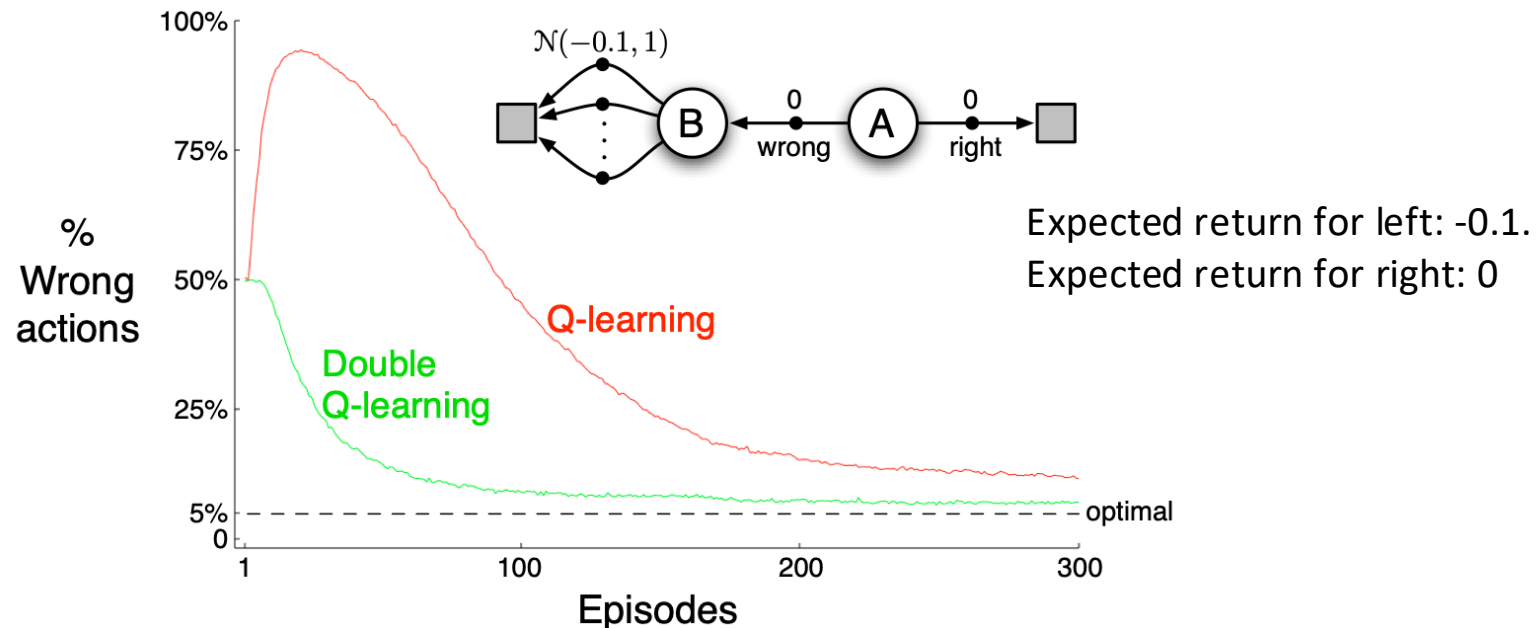
$$\square r(s, a) + \gamma \max_{a'} Q_{\phi^-}(s', a') = r + \gamma Q_{\phi^-}(s', \arg\max_{a'} Q_{\phi^+}(s', a'))$$

□ Idea: **use two Q functions for argmax and evaluations!**

□ Motivating toy example:

Sutton & Barto
Chapter 6.7

Many actions which have a stochastic reward, all drawn from the distribution:



Double DQN (DDQN)

- ❑ In practice, use current and target networks for our “two” networks!
- ❑ <3 line change in PyTorch
- ❑ Standard DQN:

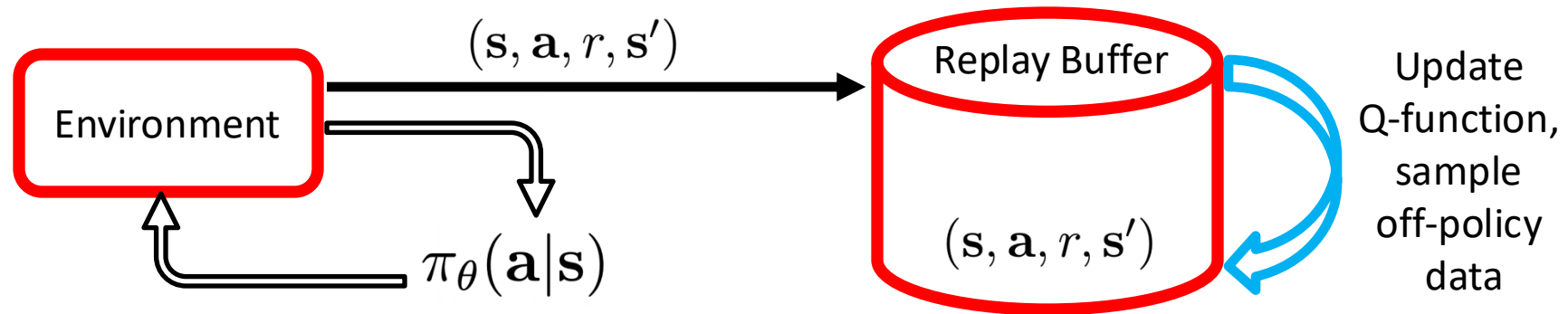
$$\mathbb{E}_{(s,a,s',r)} \left[\left(Q_{\phi}(s,a) - r(s,a) - \gamma \max_{a'} Q_{\phi^-}(s',a') \right)^2 \right]$$

- ❑ DDQN:

$$\mathbb{E}_{(s,a,s',r)} \left[\left(Q_{\phi}(s,a) - r(s,a) - \gamma Q_{\phi^-}(s', \operatorname{argmax}_{a'} Q_{\phi}(s',a')) \right)^2 \right]$$

Prioritized Experience Replay

- ❑ DDQN/DQN: all data points in the buffer are “equal”



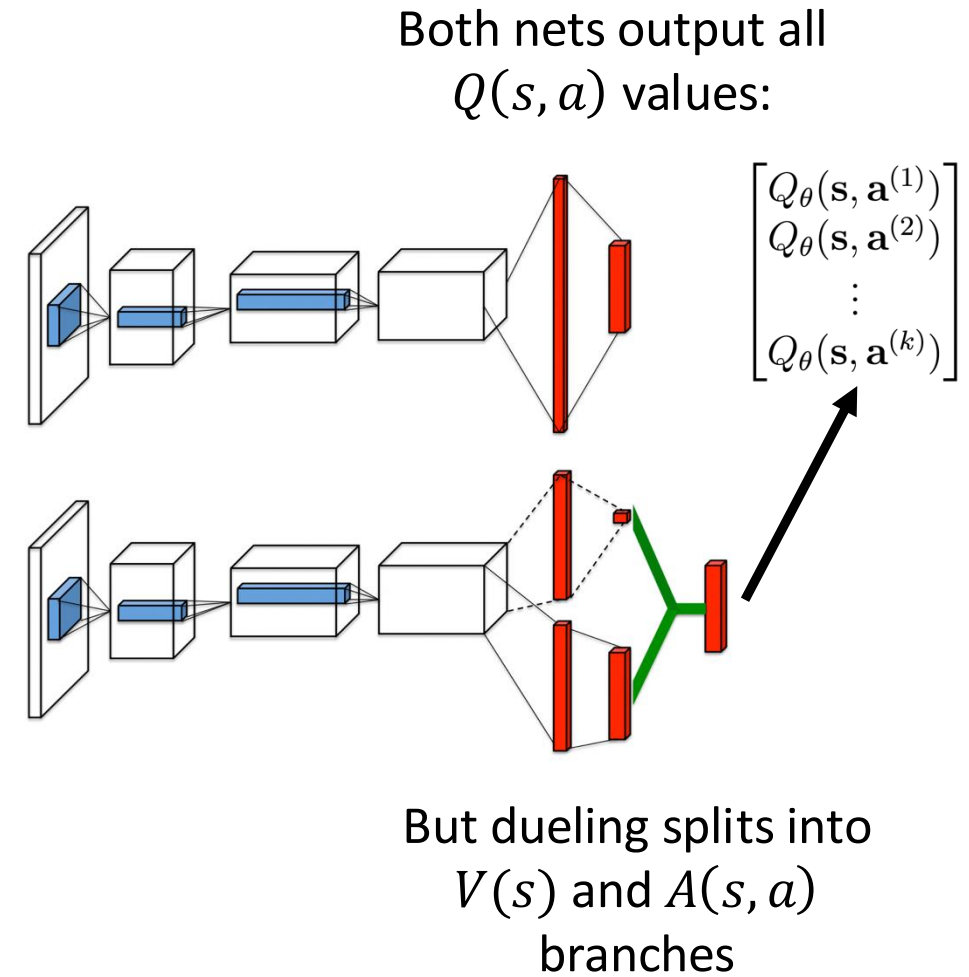
- ❑ Key idea: *prioritize* each data transition by (absolute) TD (temporal difference) error:

$$\delta = \left| Q_{\phi}(s, a) - r(s, a) - \gamma Q_{\phi}(s', \underset{a'}{\operatorname{argmax}} Q_{\phi}(s', a')) \right|$$

- ❑ Intuition: δ seems to be a reasonable approximation to the “usefulness” of a sample
- ❑ Then sample transitions with bigger δ more frequently
- ❑ δ gets updated whenever the transition is sampled.

Dueling Architectures for DQN

- ❑ Modify Q-network architecture.
- ❑ Normally: input is state or obs, output are all $Q(s, a)$ values.
- ❑ Proposed: split into scalar state value $V(s)$ and **advantages** $A(s, a) = Q(s, a) - V(s)$ for all actions, then combine.
- ❑ Train using standard DQN settings.
- ❑ Why? Means the agent does not need to learn $A(s, a)$ or $Q(s, a)$ for all states and all actions; useful if many states exist where action choices won't matter much.



Q-Learning with Continuous Actions

❑ How to solve max or argmax with continuous actions?

$$\mathbb{E}_{(s,a,s',r)} \left[\left(Q_{\phi}(s,a) - r(s,a) - \gamma Q_{\phi}(s', \underset{a'}{\operatorname{argmax}} Q_{\phi}(s', a')) \right)^2 \right]$$

❑ Option 1: **discretization**

- Not scalable

❑ Option 2: **optimization**

- Gradient-based optimization (GD, SGD) on $Q_{\phi}(s', a')$
- Stochastic optimization
 - Cross-entropy method (CEM): simple iterative stochastic optimization
 - CMA-ES: more complex iterative stochastic optimization
- Problem: slow and stuck in the local minimum!

Q-Learning with Continuous Actions

□ How to solve max or argmax with continuous actions?

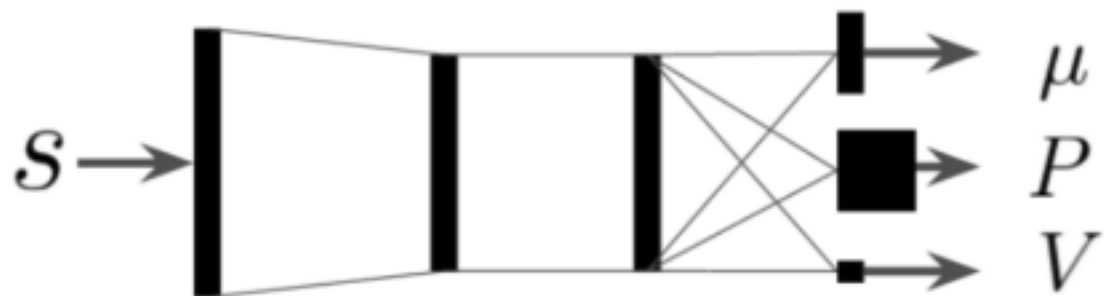
$$\mathbb{E}_{(s,a,s',r)} \left[\left(Q_{\phi}(s,a) - r(s,a) - \gamma Q_{\phi^{-}}(s', \underset{a'}{\operatorname{argmax}} Q_{\phi}(s', a')) \right)^2 \right]$$

□ Option 3: **structured Q functions that are easy to optimize**

- Example: NAF (normalized advantage functions)
- Problem: less expressive power!

$$Q_{\phi}(\mathbf{s}, \mathbf{a}) = -\frac{1}{2}(\mathbf{a} - \mu_{\phi}(\mathbf{s}))^T P_{\phi}(\mathbf{s})(\mathbf{a} - \mu_{\phi}(\mathbf{s})) + V_{\phi}(\mathbf{s})$$

$$\underset{\mathbf{a}}{\operatorname{argmax}} Q_{\phi}(\mathbf{s}, \mathbf{a}) = \mu_{\phi}(\mathbf{s}) \qquad \underset{\mathbf{a}}{\operatorname{max}} Q_{\phi}(\mathbf{s}, \mathbf{a}) = V_{\phi}(\mathbf{s})$$



NAF Architecture.

Q-Learning with Continuous Actions

□ How to solve max or argmax with continuous actions?

$$\mathbb{E}_{(s,a,s',r)} \left[\left(Q_{\phi}(s,a) - r(s,a) - \gamma Q_{\phi}(s', \underset{a'}{\operatorname{argmax}} Q_{\phi}(s', a')) \right)^2 \right]$$

□ Option 4: **learn an approximate optimizer**

□ Example: DDPG (deep deterministic policy gradient, Lillicrap et al, ICLR'16)

- Main idea: train another DNN $\mu_{\theta}(s)$ such that $\mu_{\theta}(s) \approx \underset{a}{\operatorname{argmax}} Q_{\phi}(s, a)$
- $\mu_{\theta}(s)$ is essentially a **policy**!
- How? Gradient ascent on $Q_{\phi}(s, \mu_{\theta}(s))$! Chain rule: $\frac{dQ}{d\theta} = \frac{dQ}{da} \frac{d\mu}{d\theta}$
- It is a deterministic actor-critic algorithm (more details in future lecture), but really can be viewed as an approximate Q learning with continuous action space

Q-Learning with Continuous Actions

□ DDPG (deep deterministic policy gradient, Lillicrap et al, ICLR'16)

for $t = 1, T$ **do**

Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

Execute action a_t and observe reward r_t and observe new state s_{t+1}

Store transition (s_t, a_t, r_t, s_{t+1}) in R

Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

Update the actor policy using the sampled policy gradient:

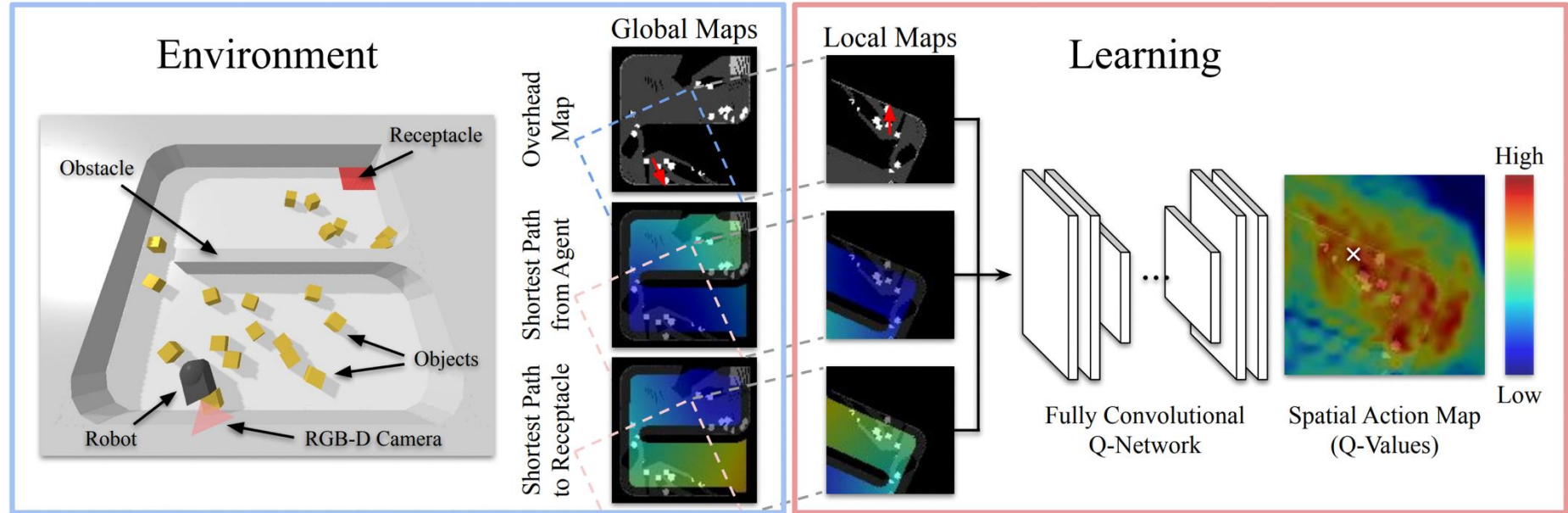
$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

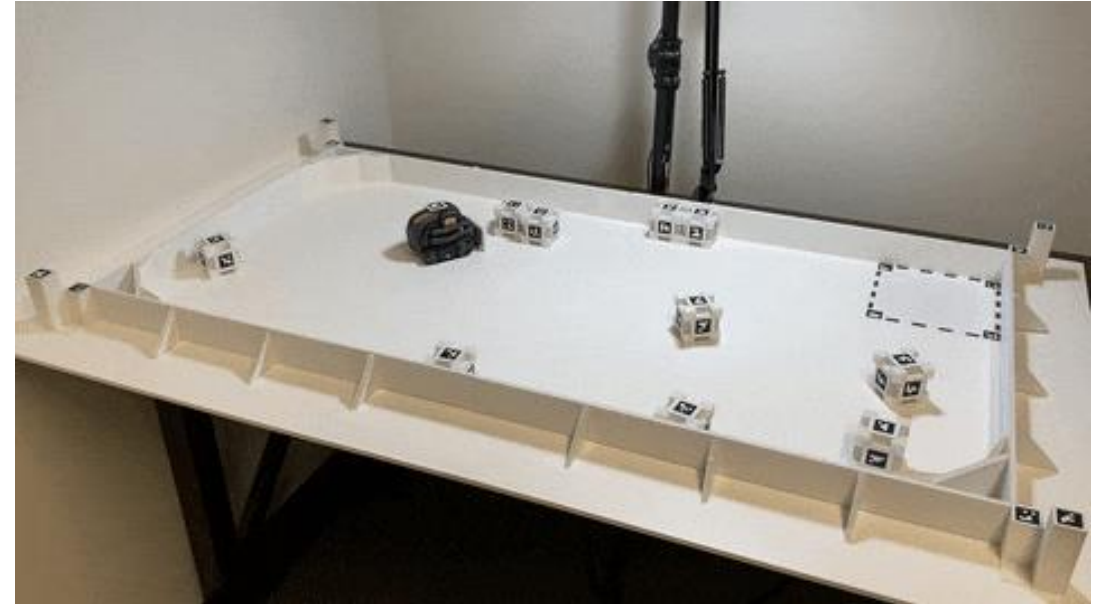
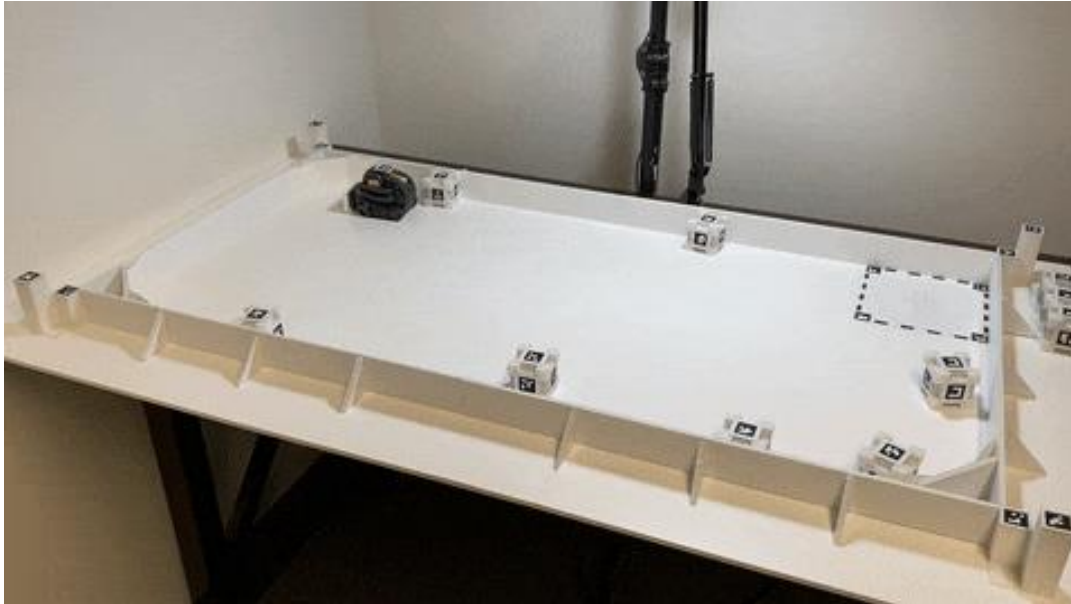
$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$$

Q-Learning in Robotics



- ❑ Actions represented as Spatial Action Maps showing the estimated Q-value if the agent were to move to the pixel (endpoint).
 - Similar to the segmentation problem
 - It is why the FCN architecture is used

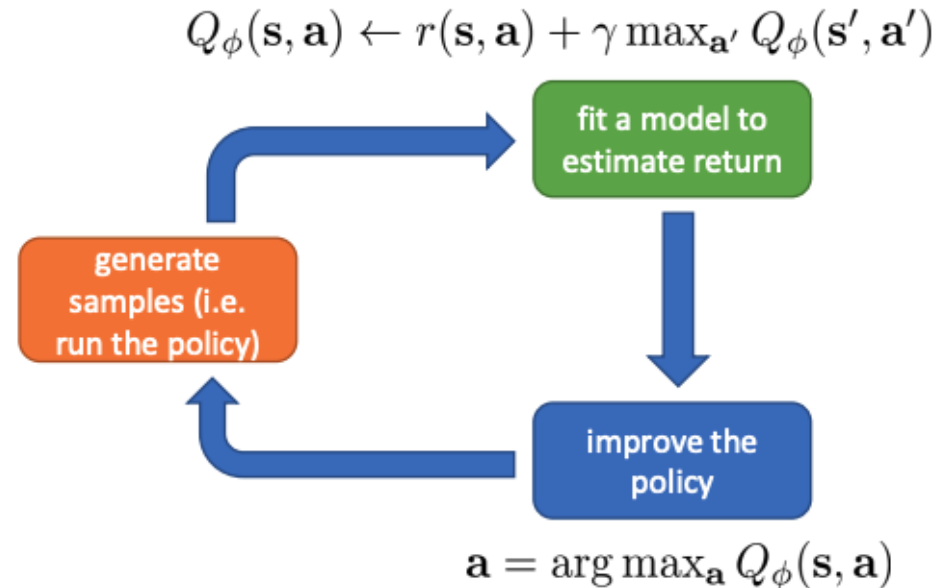
Q-Learning in Robotics



- ❑ Learned policy can exhibit long-term behavior
 - Push blocks against wall and using wall to help push multiple items

Summary of Q-Learning

❑ Key idea:



- ❑ Full offline: fitted Q iteration
- ❑ Online with neural network approximations: DQN and variants
- ❑ Q-learning is **off-policy**
- ❑ Important algorithmic ideas: replay buffer, target network, double Q learning, dueling, prioritized buffer
- ❑ Q-learning in continuous action space

Thank You!